

Minor Project

 **Task3:** Malware Detection tool

 **Deadline:** 15th Feb, 11:59 PM

Malware:

Malware is software designed to harm, exploit, or compromise a system.

Example include:

- ➔ Viruses
- ➔ Worms
- ➔ Trojans
- ➔ Ransomware
- ➔ Spyware

Malware Detection Tool:

A malware detection tool scans files, applications, or systems to detect malicious threats. It does this by:

- Checking **file signatures** against a known malware database.
- Monitoring **suspicious behavior** (e.g., unauthorized access).
- Detecting **anomalies in system activity**.

Hash Signatures

A **hash signature** is a unique, fixed-length code generated using a **cryptographic hashing algorithm** (such as SHA-256) from a file's contents. It acts like a **fingerprint** for a file.

Why SHA-256?

- Collision-resistant

- **Highly secure (256-bit encryption)**
- **Widely trusted**
- **Ideal for cryptographic security and malware detection**

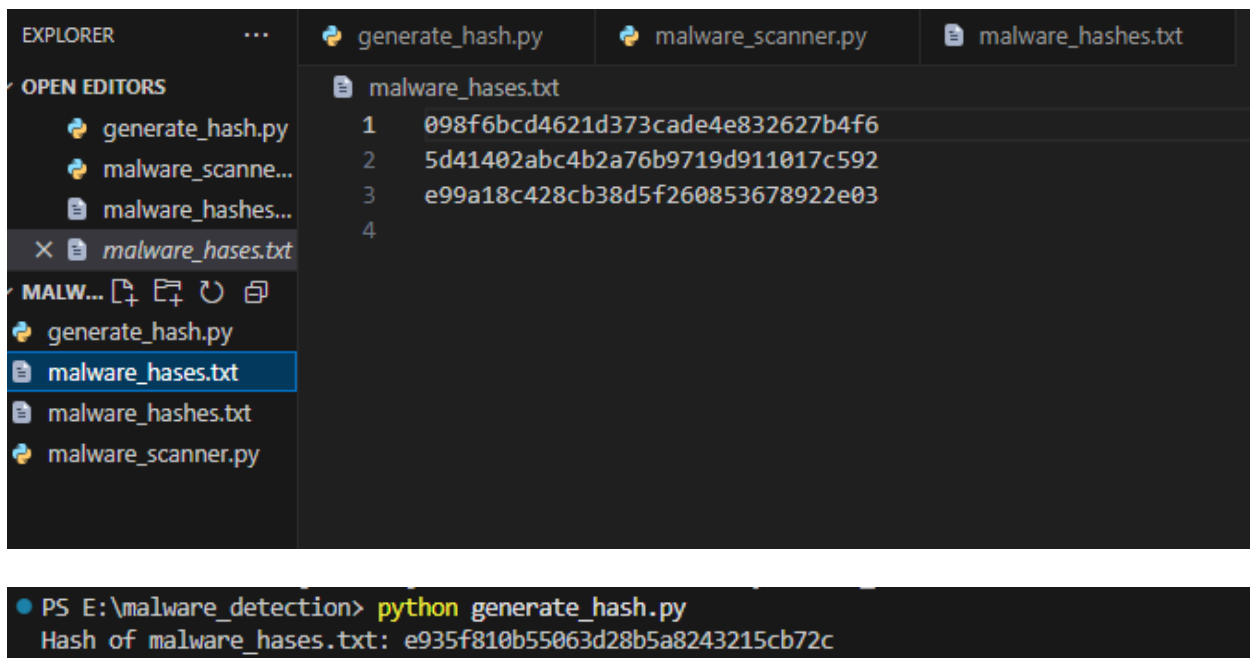
Project structure:

malware_detection

```
| — generate_hash.py    # Script to generate file hashes
| — malware_hashes.txt  # List of known malware hashes
| — malware_scanner.py  # Malware detection scanner
| — README.md           # Documentation
```

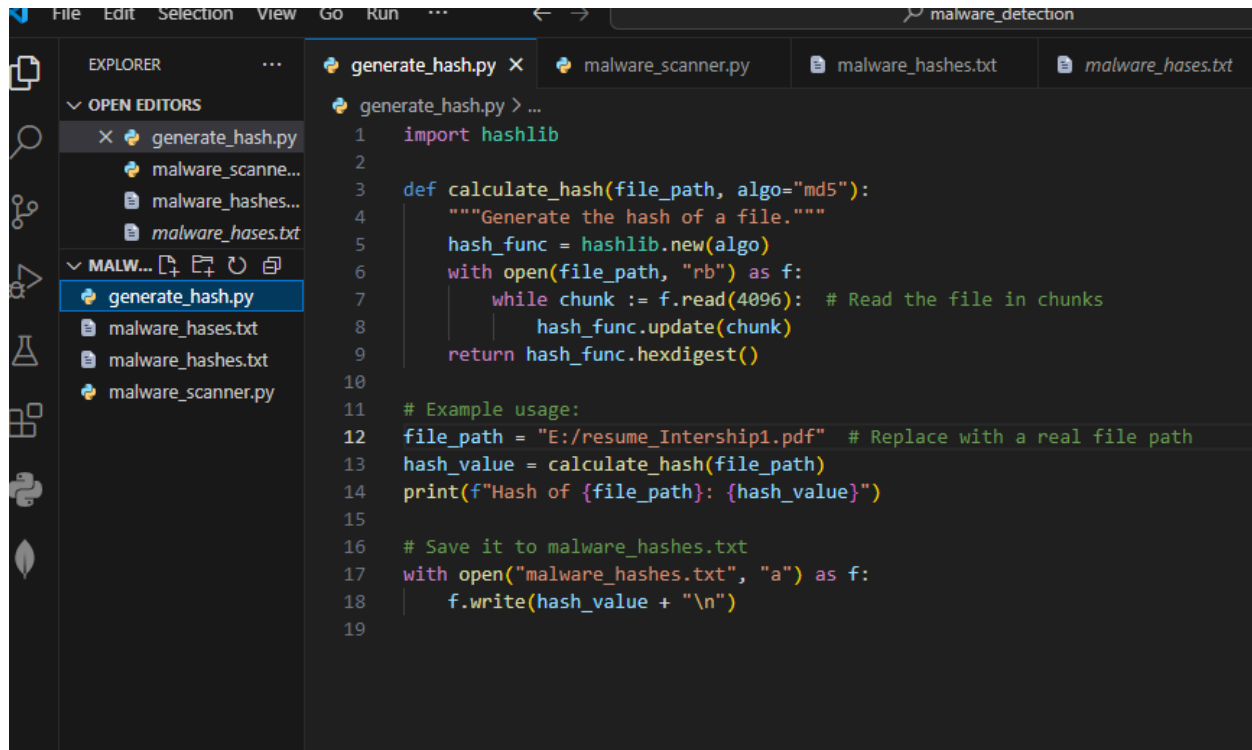
Step 1: creating malware_hases.txt file

The first step is to create a file named `malware_hashes.txt`, which will store hash signatures of known malware files.



Step 2: Generating Hashes for Files:

We created a script `generate_hash.py` to generate SHA-256 hashes for any file

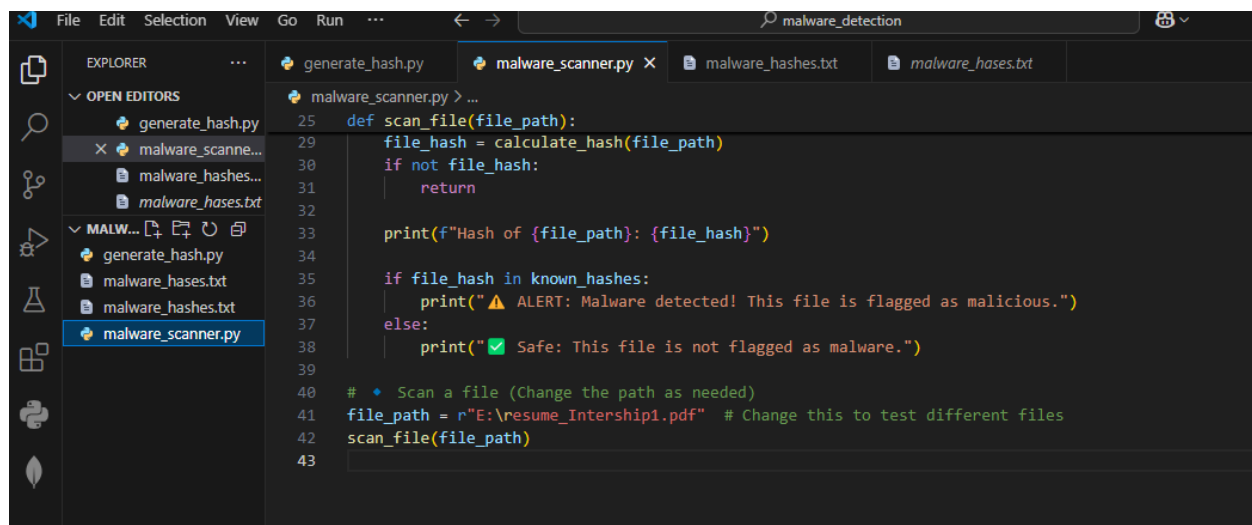


The screenshot shows a code editor with the file `generate_hash.py` open. The Explorer panel on the left shows the project structure with files `generate_hash.py`, `malware_scanner.py`, `malware_hashes.txt`, and `malware_hases.txt`. The main editor displays the following Python code:

```
1 import hashlib
2
3 def calculate_hash(file_path, algo="md5"):
4     """Generate the hash of a file."""
5     hash_func = hashlib.new(algo)
6     with open(file_path, "rb") as f:
7         while chunk := f.read(4096): # Read the file in chunks
8             hash_func.update(chunk)
9     return hash_func.hexdigest()
10
11 # Example usage:
12 file_path = "E:/resume_Intership1.pdf" # Replace with a real file path
13 hash_value = calculate_hash(file_path)
14 print(f"Hash of {file_path}: {hash_value}")
15
16 # Save it to malware_hashes.txt
17 with open("malware_hashes.txt", "a") as f:
18     f.write(hash_value + "\n")
19
```

Step 3: Scanning for Malware:

then created `malware_scanner.py`, which compares the hash of a file with those stored in `malware_hashes.txt`.



The screenshot shows a code editor with the file `malware_scanner.py` open. The Explorer panel on the left shows the project structure with files `generate_hash.py`, `malware_scanner.py`, `malware_hashes.txt`, and `malware_hases.txt`. The main editor displays the following Python code:

```
25 def scan_file(file_path):
26     file_hash = calculate_hash(file_path)
27     if not file_hash:
28         return
29
30     print(f"Hash of {file_path}: {file_hash}")
31
32     if file_hash in known_hashes:
33         print("⚠️ ALERT: Malware detected! This file is flagged as malicious.")
34     else:
35         print("✅ Safe: This file is not flagged as malware.")
36
37 # • Scan a file (Change the path as needed)
38 file_path = r"E:\resume_Intership1.pdf" # Change this to test different files
39 scan_file(file_path)
40
```

Step 4: Running the Malware Scanner

- Run `generate_hash.py` to get the hash of a file.
- Run `malware_scanner.py` to check if the file is malware.

```
PS E:\malware_detection> python generate_hash.py
Hash of E:/resume_Intership1.pdf: 7f155beb76a0663210d7f8d9fed7f566
PS E:\malware_detection> python malware_scanner.py
>>
Hash of E:\resume_Intership1.pdf: 7f155beb76a0663210d7f8d9fed7f566
⚠️ALERT: Malware detected! This file is flagged as malicious.
PS E:\malware_detection> python malware_scanner.py
Hash of E:\resume_Intership1.pdf: 7f155beb76a0663210d7f8d9fed7f566
✅ Safe: This file is not flagged as malware.
PS E:\malware_detection> 
```

☑ Observations and Recommendations

- The scanned system has common web services running (HTTP, HTTPS).
- No immediate vulnerabilities detected, but regular security patches are recommended.
- Secure SSH by restricting access and using key-based authentication.
- Firewall rules should be reviewed to ensure no unnecessary open ports.

🎯 Conclusion

Network scanning with Nmap provided insights into the active hosts, open ports, and running services. The scan results highlight potential security measures that can be implemented to secure the network effectively.

📄 **Submitted By:** [Deepika Deshmukh]

📅 **Date:** 16th Feb

📁 **Submission:** WEE4 Day 3

